CHAPITRE

Héritage et Polymorphisme

03

Objectifs d'apprentissage

- Comprendre les concepts fondamentaux d'**héritage** et de relation "est-un".
- Maîtriser la redéfinition avec `@Override` et l'usage de `super` et `final`.
- Concevoir des hiérarchies robustes grâce au **polymorphisme** statique et dynamique.

CHAPITRE 3.1

L'Héritage : Définition et Concepts

L'héritage est un mécanisme fondamental de la POO permettant à une classe d'acquérir les propriétés et comportements d'une autre classe.

Relation "Est-un" (Is-A)

Établit un lien sémantique fort. Un `Chien` est un `Animal`.

Réutilisation du Code

Évite la duplication. Les attributs et méthodes communs sont définis une seule fois dans la classe mère.

Extension des Fonctionnalités

Permet d'enrichir ou de modifier le comportement existant sans altérer le code original.

Organisation Hiérarchique

Structure le code de manière logique, facilitant la maintenance et la compréhension du système.



Note: La classe fille hérite automatiquement de `nom`, `manger()` et `dormir()`.

CHAPITRE 3.1

Syntaxe et mot-clé super

Animal.java

```java
public class Animal {
  protected String nom;
  public Animal(String nom) {
    this.nom = nom;
  }
  public void manger() { ... }
}
```

Chien.java

```java
public class Chien extends Animal {
  private String race;
  public Chien(String nom, String race) {
    // Appel constructeur parent (OBLIGATOIRE)
    super(nom);
    this.race = race;
  }
}
```

L'héritage avec `extends`

Le mot-clé `extends` établit la relation d'héritage. La classe fille (Chien) récupère automatiquement tous les attributs et méthodes `public` et `protected` de la classe mère (Animal).

Le constructeur `super()`

La classe fille **doit** appeler un constructeur de la classe parente. L'instruction `super(...)` doit impérativement être la **première instruction** du constructeur enfant.

Accès aux membres `super.`

Le mot-clé `super` permet aussi d'accéder explicitement aux membres de la classe parente, notamment pour appeler une méthode parente qui a été redéfinie (ex: `super.manger()`).

CHAPITRE 3.1

Accès et mot-clé `final`

Niveaux de visibilité

`private`	Visible uniquement dans la **classe** elle-même.
`default`	Visible dans tout le **package** (absence de mot-clé).
`protected`	Visible par **package** + **sous-classes**.
`public`	Visible **partout** dans l'application.

Usage de `final`

UsageFinal.java

```java
// 1. Empêcher l'héritage
public final class Securite { ... }
  // ERREUR: class X extends Securite
public class Compte {
  // 2. Empêcher la redéfinition
  public final void valider() {
    System.out.println("Validé");
  }
}
```

Pourquoi utiliser final ?

Sécurité : Garantit que des comportements critiques (ex: authentification) ne sont pas altérés par des sous-classes.

Conception : Indique qu'une classe n'est pas conçue pour être étendue (ex: classe `String`).

CHAPITRE 3.1.5

Redéfinition de Méthodes (Override)

Oiseau.java

```java
public class Oiseau extends Animal {
  private boolean peutVoler;
  // Constructeur ...
  @Override
  public void manger() {
    super.manger(); // Optionnel
    System.out.println("Picore des graines");
  }
  // ERREUR : Impossible de redéfinir final
  /*
  @Override
  public void respirer() { ... }
  */
}
```

L'annotation @Override force la vérification du compilateur

Règles de Redéfinition

Signature Identique

Le nom et les paramètres doivent correspondre exactement à la méthode parente.

Visibilité (Scope)

Ne peut pas être plus restrictive. `public` → `protected` est interdit.

Covariance

Le type de retour peut être une sous-classe du type de retour original (ex: Animal → Chien).

Restrictions Importantes

On ne peut pas redéfinir les méthodes déclarées `final` ou `static`.

ÉTUDE DE CAS

Système Bancaire



↑ Le polymorphisme permet de traiter ces objets via le type parent

Analyse de la Conception

Comment l'héritage et le polymorphisme résolvent le problème métier.

Réutilisation du Code

La méthode `deposer()` est implémentée une seule fois dans la classe mère. Tous les comptes en bénéficient sans duplication.

Logique Métier Spécifique

La méthode `retirer()` est redéfinie pour appliquer des règles distinctes : interdiction de découvert pour l'épargne, autorisation limitée pour le courant.

Usage Polymorphe

Le système peut gérer une liste de `List<CompteBancaire>` et appeler `retirer()` sans connaître le type exact du compte.

CHAPITRE 3 (SUITE)

Le Polymorphisme

Objectifs de la section

- Distinguer la **surcharge** (statique) de la **redéfinition** (dynamique).
- Maîtriser l'opérateur `instanceof` et le casting d'objets sécurisé.
- Manipuler des **collections polymorphes** pour un code plus générique.

CHAPITRE 3.2

Le Polymorphisme

Du grec "plusieurs formes", c'est la capacité d'un objet à prendre plusieurs formes et d'une méthode à avoir plusieurs comportements selon le contexte.

Polymorphisme Dynamique (Runtime)

Basé sur la **redéfinition** (Override) de méthodes.

La méthode exécutée est déterminée lors de l'exécution selon le type réel de l'objet.

Polymorphisme Statique (Compile-time)

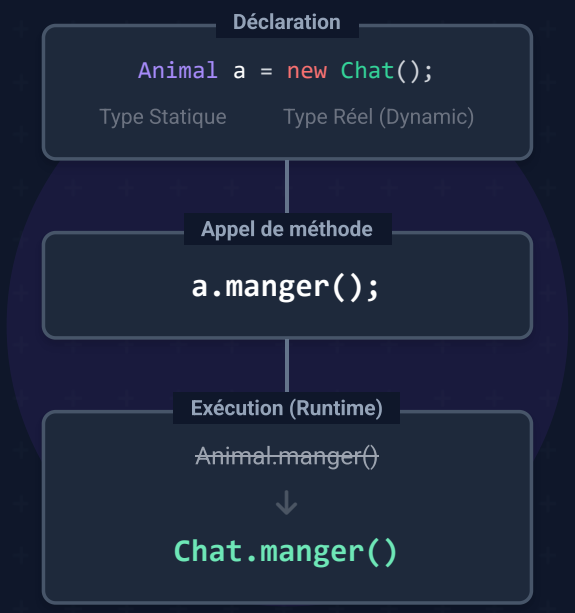
Basé sur la **surcharge** (Overloading) de méthodes.

Le choix de la méthode se fait à la compilation selon les types des arguments.

Abstraction & Généricité

Permet d'écrire du code flexible qui manipule des objets via leur type parent (ex: `List<Animal>`) sans connaître leur type spécifique.

RÉSOLUTION DYNAMIQUE



"Bien que la référence soit de type Animal, c'est la méthode du Chat qui est exécutée."

CHAPITRE 3.2.2

Polymorphisme par Héritage

TestPolymorphisme.java

```java
public static void main(String[] args) {
  // Référence parent = Objet enfant
  Animal a1 = new Chien("Rex", 3);
  Animal a2 = new Chat("Félix", 2);
  // RÉSOLUTION DYNAMIQUE (Runtime)
  a1.manger(); // → Chien.manger()
  a2.manger(); // → Chat.manger()
  // ERREUR DE COMPILATION
  // Animal ne connait pas aboyer()
  // a1.aboyer();
}
```

>_ Sortie: Rex (Chien) mange... / Félix (Chat) mange...

Mécanismes Clés

Type Déclaré vs Type Réel

La variable est de type Animal (parent), mais l'objet en mémoire est de type Chien (enfant). C'est le polymorphisme.

Liaison Dynamique

À l'exécution, la JVM cherche la version de la méthode correspondant au **type réel** de l'objet (ici Chien), pas au type de la variable.

Sécurité du Typage

Le compilateur vérifie uniquement si la méthode existe dans la classe Animal.

Attention aux méthodes spécifiques

Impossible d'appeler `aboyer()` sur une référence Animal sans faire un cast explicite.

CHAPITRE 3.2.3

L'opérateur `instanceof` et Casting

TypeCheck.java

```java
public void traiterAnimal(Animal animal) {
  // Appel polymorphe (sûr pour tous)
  animal.manger();
  // Vérification avant Transtypage (Casting)
  if (animal instanceof Chien) {
    Chien chien = (Chien) animal;
    chien.aboyer(); // Méthode spécifique
  }
  // Pattern Matching (Java 16+)
  else if (animal instanceof Chat chat) {
    chat.miauler(); // Cast automatique !
  }
}
```

</> Pattern matching simplifie la syntaxe

Mécanismes de Transtypage

Vérification de Type

L'opérateur `instanceof` vérifie si un objet est une instance de la classe cible (ou d'une sous-classe). Retourne `true` ou `false`.

Downcasting (Transtypage vers le bas)

Nécessaire pour redonner à une référence parent (`Animal`) son type spécifique (`Chien`) et accéder à ses méthodes uniques.

Pattern Matching Java 16+

Fusionne la vérification et le casting en une seule expression, réduisant le code boilerplate et les erreurs.

Attention : ClassCastException

Forcer un cast sans vérification (ex: caster un Chat en Chien) provoquera une erreur fatale à l'exécution.

CHAPITRE 3.2

Collections & Surcharge

Polymorphisme avec Collections

ZooSimulation.java

```java
// Liste typée avec la classe parente
List<Animal> zoo = new ArrayList<>();
zoo.add(new Chien("Rex"));
zoo.add(new Chat("Félix"));
// Traitement uniforme
for (Animal a : zoo) {
  // Appel résolu à l'exécution (Runtime)
  a.manger();
}
```

Résolution Dynamique

Bien que la référence soit de type `Animal`, la JVM identifie le type réel de chaque objet en mémoire lors de l'exécution.

Résultat : Le Chien mange comme un chien, le Chat comme un chat.

Surcharge (Overloading)

Calculatrice.java

```java
public class Calculatrice {
  // Signature 1 : entiers
  int add(int a, int b) { ... }
  // Signature 2 : double
  double add(double a, double b) { ... }
  // Signature 3 : trois paramètres
  int add(int a, int b, int c) { ... }
}
```

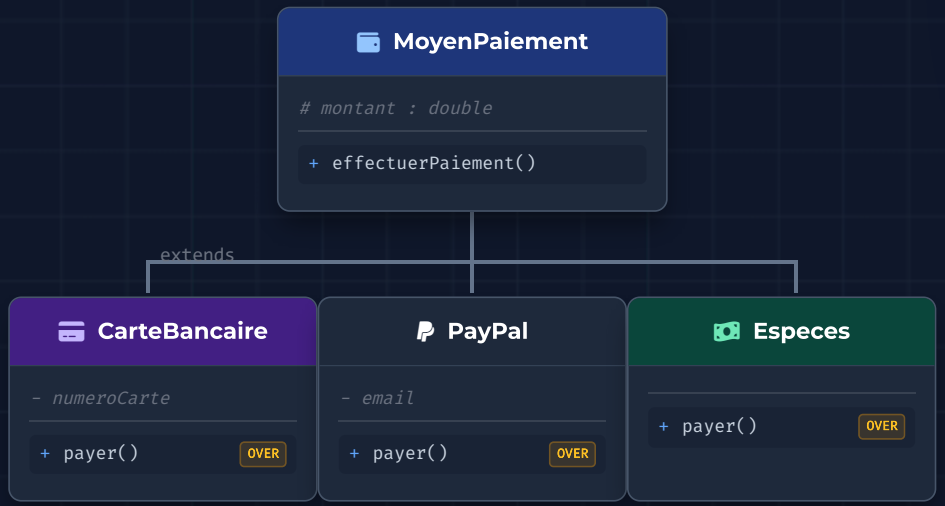
Résolution Statique

Le compilateur choisit la méthode à appeler en fonction du **nombre** et du **type** des arguments passés.

Ce n'est pas lié à l'héritage, mais à la signature de la méthode.

ÉTUDE DE CAS

Système de Paiement



La classe `CaisseEnregistreuse` appelle `paiement.effectuerPaiement()` sans connaître le type réel.

Avantages du Design

Pourquoi utiliser le polymorphisme pour les paiements ?

Extensibilité (Open/Closed)

On peut ajouter `CryptoMonnaie` sans modifier le code de la caisse. Le système est ouvert à l'extension mais fermé à la modification.

Découplage Fort

La logique de traitement des paiements (validation, connexion API) est encapsulée dans chaque sous-classe, pas dans le flux principal.

Uniformité de l'API

Toutes les méthodes de paiement respectent le même contrat. Le code client est simplifié et plus lisible.

CHAPITRE 3.2.6

Exemple Complet | Système de Paiement

CaisseEnregistreuse.java

CaisseEnregistreuse.java

```java
public class CaisseEnregistreuse {
  // Méthode polymorphe : accepte n'importe quel sous-type
  public void traiterPaiement(MoyenPaiement paiement) {
    System.out.println("--- Traitement ---");
    // Appel dynamique (Late Binding)
    paiement.effectuerPaiement(); // ⟵ Le cœur du polymorphisme
    System.out.println("Paiement validé!\n");
  }
  public static void main(String[] args) {
    CaisseEnregistreuse caisse = new CaisseEnregistreuse();
    // Injection des dépendances spécifiques
    MoyenPaiement cb = new CarteBancaire(50.0, "1234 ... ");
    MoyenPaiement pp = new PayPal(75.50, "user@mail.com");
    caisse.traiterPaiement(cb); // Appelle version Carte
    caisse.traiterPaiement(pp); // Appelle version PayPal
  }
}
```

Abstraction

La méthode `traiterPaiement` ne connaît que la classe mère `MoyenPaiement`. Elle est découplée des implémentations spécifiques (Carte, PayPal, etc.).

Liaison Dynamique

À la ligne 8, la JVM décide au moment de l'exécution quelle méthode appeler (celle de `CarteBancaire` ou de `PayPal`) en fonction du type réel de l'objet.

Extensibilité

On peut ajouter une classe `CryptoMonnaie` sans modifier une seule ligne de code dans la classe `CaisseEnregistreuse`.